Sim-to-Real Augmented Policies Under Model Uncertainty

Ryan Draves
University of Colorado Boulder
Smead Aerospace Engineering Sciences
Boulder, CO

***Index Terms*—Model Uncertainty, Structured Latent Spaces, Sim-to-Real Transfer, Augmented Policies**

***Abstract*—Sim-to-real transfers are often encumbered by discrepancies between simulated models and real environments. This work investigates two experiments where simple policies designed for idealized models are augmented to operate in more complex environments. In the first experiment, a Tic-Tac-Toe policy is adapted to an expanded environment with anomalous obstacles through a structured latent space and a hybrid policy. In the second experiment, a two wheeled robot controller is augmented to account for unmodeled motor degradations and drag forces through supervised reinforcement learning. Across both experiments, we show that simple policies can be effectively adapted to more complex environments through learned augmentations.**

I. Introduction

As advancements are made in the development of planning under uncertainty, a frequent trouble in robotic applications continues to be the transfer of techniques developed in simulated environments to the real world. The issue primarily lies in the discrepancy between the idealized environment of a simulation versus the practical considerations of reality, resulting in a mismatch between the model a policy believes it is acting in (or was meant to) and the environment a policy is truly acting in. There are uncountably many reasons that could be attributed to these mismatches; perhaps external factors aren't accounted for, such as the busyness of the scenery a robot senses or the unrelenting inaccuracies when obeying a wall clock. Perhaps the dynamics of the model are wrong, not accounting for bias in the slight manufacturing differences between the robot's motors or the strong crosswind in today's weather. Or less insidiously, perhaps a policy expected to act on an abstracted, simplified state space when only a raw camera image is available as input.

In this work, we investigate a few methods to deal with model discrepancies while still allowing for simple policies designed in simulated environments to be employed. In our first experiment, a simple Tic-Tac-Toe policy will be harnessed in an environment given by image inputs and an expanded state space represented by anomalous blobs. A *structured latent space* will allow the simple policy to be invoked when the anomaly is observed to be absent and a learned supplemental policy will be invoked when the anomaly is observed to be present. In the second experiment, a two wheeled robot is manipulated by a controller given an idealized understanding of the car's dynamics. When employed in a more complicated environment, an augmented policy learns to adapt the controller's outputs to better align the "physical" realizations of its actions with the simulated realizations the controller believed would happen in its idealized environment. Across these experiments, we show that machine learning techniques can be used to harness simple policies designed for idealized versions of the environments they are employed in.

II. Related Work

In the context of latent space research, several previous works have explored disentangled latent spaces where the core, "independent" dimensions of a problem are separated out in the latent representation. For example in the popular MNIST handwriting dataset, one may want a latent representation where one dimension is the line width and the other is the slant of the digit. Jun et al [1] present a recent advancement in disentangled representation learning where these "intrinsic factors" are separated out in the latent space when using diffusion model architectures[2]. InfoGAN [3] learns to create disentangled representations from Generative Adversarial Networks (GANs) [4]. Building on variational autoencoders (VAEs)[5], where the latent space is represented by a distribution, β-VAE augments the learning process to encourage learning statistically inde-

pendent latent factors within the distribution[6]. Each of these methods utilize unsupervised learning techniques to create disentangled representations.

Closer to this work's first experiment use of latent spaces, Le et al propose the supervised autoencoder [7] in which input data is labelled with a corresponding expected output in the latent space and the representation loss of the autoencoder is combined with a loss on the accuracy of the latent output. A similar technique will be employed in the the "structured" part of this work's latent space. Zhang et al [8] propose a method of learning "task-relevant details" from images, discarding "task irrelevant" information from its encoder and enabling tasking over generalized domains.

Related to the second experiment, Lange et al [9] employed autoencoders with a form of deep Q learning to create an early work on end-to-end RL agents in robotic applications. Several works build on this idea to learn dynamics models within these latent spaces, i.e. learning an abstracted model of the environment that is propagated within the latent space to separate learning a model of the environment from learning a policy to act within that modeled environment [8, 10–12]. Some of these works [8, 11] touch on bisimulation literature to quantify how "close" the learned models are to the environments they abstract.

Lastly, prominent sim-to-real literature such as Peng et al [13] employ domain randomization to generalize an agent trained in simulation to real environments. Two works on residual policy learning [14, 15] augment policies in a similar manner as the approach in the second experiment, but do so through the lens of improving a policy's expected reward within a single model of the environment.

III. Problem Formulation

A. Tic-Tac-Oh-No

Our first experiment, an expanded version of Tic-Tac-Toe, will have a static policy for Player 2 given by winning actions, when available, and random actions otherwise. No-op actions are taken in the presence of the anomaly, leaving Player 1 to deal with it. With this static policy, our Tic-Tac-Oh-No game will be reduced to an MDP where we expect Player 1 to win roughly two thirds of games and draw the rest. A two player game was chosen for its easy visualization, rather than the investigation of two player games.

1) Nominal Model: Our nominal model of Tic-Tac-Oh-No (i.e., standard Tic-Tac-Toe) is an MDP where the state space is a 3×3 occupancy grid with entries in $\{-1, 0, 1\}$, the action space consists of Player 1 selecting an empty grid cell, transitions are given by Player 1's action followed by Player 2's response, and rewards are assigned for Player 1 winning the game.

The nominal policy is given by a simple deep Q reinforcement learning network with a single linear hidden layer. The results of its training are shown in Figure 1. In validation, the policy shows that it can exploit Player 2's policy and win two thirds of games and draw the rest.

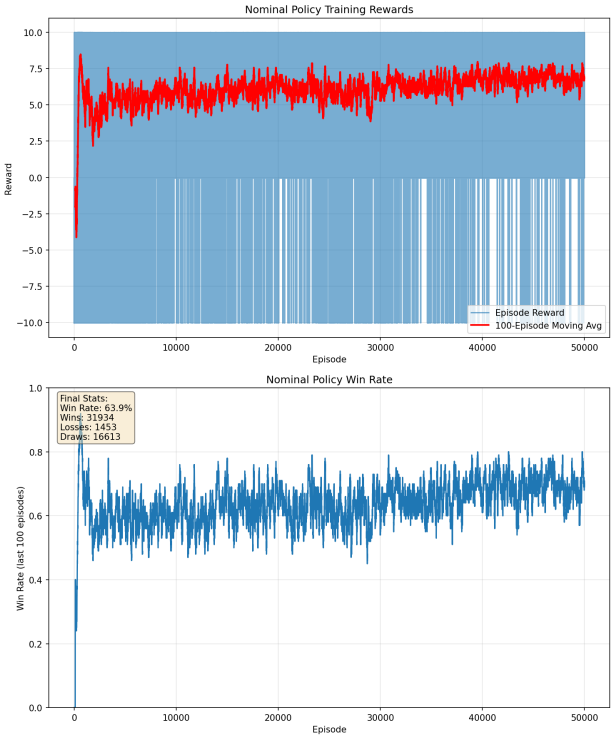

Fig. 1. Tic-Tac-Toe policy training.

2) Environment: In our full environment, the game of Tic-Tac-Toe is expanded by the presence of an "anomalous blob" which may appear on a random part of the grid after Player 1's moves. The game of Tic-Tac-Toe cannot continue for either player until dedicated actions are taken to remove the anomaly, shrinking Player 1's expected future reward in the process. Furthermore, the game is observed by Player 1 as a 60×60 pixel image rather than a direct observation of the grid state. This new environment could either be viewed as an MDP over the game pixels as the state space or a POMDP over the underlying state observed through the pixels; being fully observable at each step, it makes no difference. Figure 2 visualizes this game at a state when the anomaly appears. The 60×60 image passed to the agent is approximately a downsample of this reference visualization.

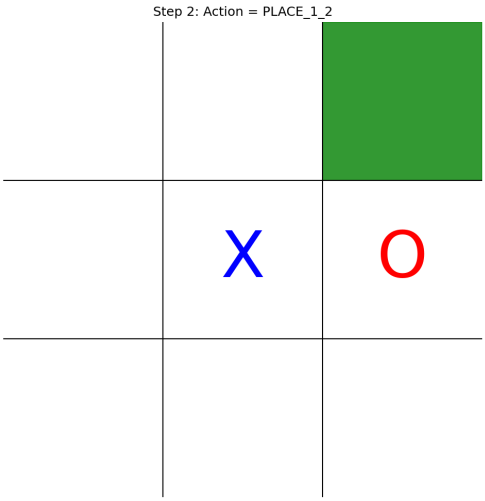

Fig. 2. Tic-Tac-Oh-No with the anomaly present.

B. Tortoisebot

In the two wheeled robot experiment, the dynamics of the nominal model differ from those of its environment. Across the model and the environment, an accurate measurement of the robot's wheel radius, R_{wheel} and axle, L_{axle}, are known. Also common to both scenarios, the state space is defined to be $S = \{x, y, \theta, v_{xy}, v_{theta}\}$ and the action space is $A = \{v_{left}, v_{right}\}$, where the action velocities are the angular velocity of the wheels to set.

1) Nominal Model: In the idealized environment, we get the following state update equations:

$$\begin{aligned} x_{k+1} &= x_k + v_{xy,k}\cos(\theta_k)\Delta t \\ y_{k+1} &= y_k + v_{xy,k}\sin(\theta_k)\Delta t \\ \theta_{k+1} &= \theta_k + v_{\theta,k}\Delta t \\ v_{xy,k+1} &= \frac{v_{left} + v_{right}}{2} R_{wheel} \\ v_{\theta,k+1} &= \frac{v_{right} - v_{left}}{L_{axle}} R_{wheel} \end{aligned} \tag{1}$$

Our nominal controller is a simple proportional controller governed by the following equations:

$$\begin{aligned} \Delta x &= x_g - x \\ \Delta y &= y_g - y \\ \rho &= \sqrt{\Delta x^2 + \Delta y^2} \\ \alpha &= \text{atan2}(\Delta y, \Delta x) - \theta \\ \beta &= \theta_g - \theta - \alpha \\ v_{xy} &= K_\rho \rho \\ v_\theta &= K_\alpha \alpha + K_\beta \beta \\ v_{left} &= \frac{v_{xy} - v_\theta \frac{L_{axle}}{2}}{R_{wheel}} \\ v_{right} &= \frac{v_{xy} + v_\theta \frac{L_{axle}}{2}}{R_{wheel}} \end{aligned} \tag{2}$$

For the purposes of this experiment, we assume that all goal states have zero velocity, which is what the proportional controller will approach. Figure 3 shows part of the trajectory the controller will take to reach a sample goal state, having started pointed in the +X direction.

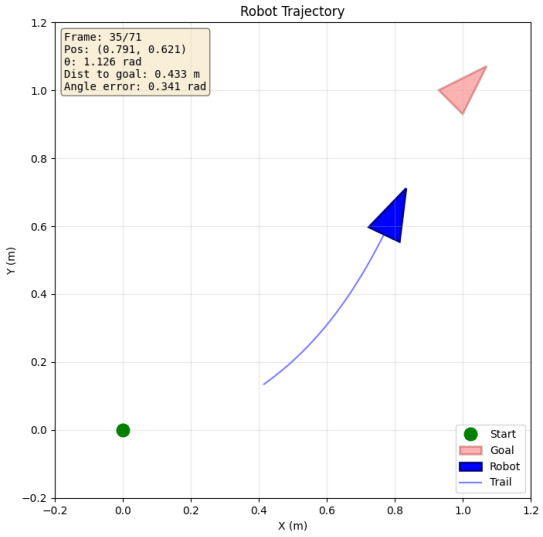

Fig. 3. Nominal tortoisebot trajectory.

2) Environment: In our full environment, the tortoisebot will encounter small motor degradations compared to the performance it envisions in the nominal model. Additionally, a simple linear drag term is introduced. These combined dynamics result in every action applied in the environment realizing slightly smaller velocity magnitudes than the same action applied in the nomi-

nal model. The modified velocity update equations are shown in Equations 3.

$$v_{xy,k+1} = (1 - d_{xy}) \frac{\alpha_{left} v_{left} + \alpha_{right} v_{right}}{2} R_{wheel}$$
$$v_{\theta,k+1} = (1 - d_{\theta}) \frac{\alpha_{right} v_{right} - \alpha_{left} v_{left}}{L_{axle}} R_{wheel} \quad (3)$$

IV. Methodology

A. Tic-Tac-Oh-No

To adopt the nominal policy into the Tic-Tac-Oh-No environment, we create a "harness" that allows the policy to be invoked on its expected inputs only when the anomaly is absent from the state of the game. This "harness" is composed of a specialized encoder and a secondary policy to be invoked in the presence of the anomaly.

1) Structured Latent Space: To train the encoder to produce outputs we can pass to the nominal policy, we partition the latent space into three parts. First, we have the grid state encoded in the first 9 dimensions. The 10th dimension is a flag for whether the anomaly is present. The remaining dimensions of the latent space (32 in total) are left to encode any information about the anomaly or input image necessary for the decoder to work. The encoder and decoder models are simple convolutional networks with a linear layer sandwiching either side of the latent space. Figure 4 visualizes this latent space, which we call a *structured latent space*.

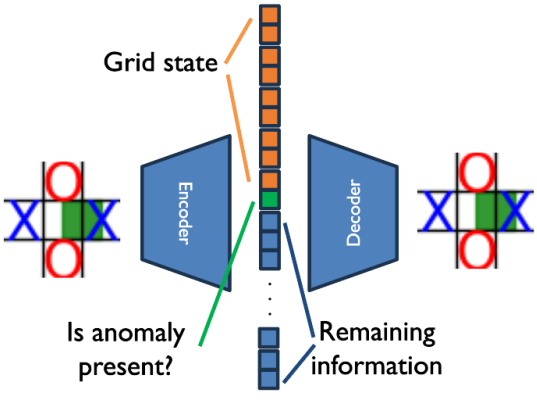

Fig. 4. Structured latent space autoencoder design.

To train the autoencoder, we combine the reconstruction loss with a MSE loss on the structured features of the first 10 latent dimensions as shown in Equations 4. Training data was generated by taking random actions in the environment, equivalent to letting an agent randomly explore in a "real-world" environment. Since detailed information about the anomaly is not part of the supervised partition of the latent space, images need only to be annotated with the game state of Tic-Tac-Toe and a flag as to whether the anomaly is present within the image.

$$\mathcal{L}_{\text{autoencoder}}(\theta) = \mathbb{E}_{x \sim p_{data}(x)} \left[\|x - f_\theta(x)\|^2 \right]$$
$$\mathcal{L}_{\text{features}}(\theta) = \mathbb{E}_{(x,y) \sim p_{data}(x,y)} \left[\|y - f_\theta(x)[:10]\|^2 \right] \quad (4)$$
$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{autoencoder}}(\theta) + \alpha \mathcal{L}_{\text{features}}(\theta)$$

2) Hybrid Policy: With the encoder producing a structured latent space familiar to the nominal policy, we devise a hybrid policy in which the nominal policy is invoked if the anomaly is observed to be absent and a secondary policy is invoked otherwise. When invoking the nominal policy, grid state representation in the latent space is cast to the strict integer inputs expected by the nominal policy, whereas the secondary policy receives the entire latent space as input. Figure 5 visualizes the invocation of both policies.

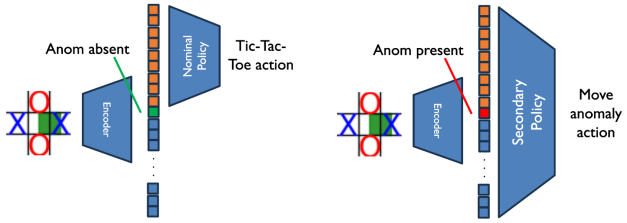

Fig. 5. Conditional invocation of nominal and secondary policy.

Since the secondary policy is invoked only in the presence of the anomaly, it is only given the actions to move the anomaly as its outputs. The secondary policy is, like the nominal policy, a simple deep Q learning network. To train it, experiences are gathered in the environment with the full hybrid policy with positive rewards attributed to clearing the anomaly off the screen and winning the game of Tic-Tac-Toe and negative rewards attributed to losing the game or the continued presence of the anomaly on the screen.

B. Tortoisebot

To bridge the dynamics gap between the simulated model and the environment, we look to augment the nominal controller with simple additions to its preferred action:

$$a = a_{nom} + f(\dots) \quad (5)$$

If we consider the simulated model and the controller to be a generator, we can produce the nominal action and

the next state the simulated model expects that action to take the robot:

$$s'_{nom}, a_{nom} = \text{nominal}(s, s_{goal}) \tag{6}$$

We design our augmented controller to use this expected next state as a reference the augmented action should get to. That is, we modify the nominal controller such that its actions realize the next state it expects to reach under the dynamics of the simulated model. The augmentations to the nominal action bridge the gap between the environment dynamics and that of the simulated model. The relative difference of positions, current and expected velocity, and the nominal actions chosen thus give us the inputs into the network that will produce the action augmentations:

$$\begin{gathered} \Delta x = \{x'_{nom} - x, y'_{nom} - y, \theta'_{nom} - \theta\} \\ v = \{v_{xy}, v_\theta\} \\ v'_{nom} = \{v'_{xy,nom}, v'_{\theta,nom}\} \\ a = a_{nom} + f(\Delta x, v, v'_{nom}, a_{nom}) \end{gathered} \tag{7}$$

We architect this network to have three hidden linear layers, each a bit larger than the previous deep Q networks with 128 dimensions. Rather than deep Q learning, we instead train with a supervised reinforcement learning approach where the agent acts in its environment to collect experiences, trying to reach randomly generated goal states. Each step of the goal trajectories is given a reward weighted by the error between the predicted next state (from the simulated model) and the actual reached next state, shown in Equation 8. These experiences are then weighted by how high their rewards are and used as supervised examples for the model to learn from. A small regularization term was also added to the supervised loss function to encourage smaller augmentations; this was found to improve performance.

$$R(s, a, s') = -100\|\vec{w}(s'_{nom} - s')\| \tag{8}$$

V. Results

A. Tic-Tac-Oh-No

1) Structured Latent Space: The encoder learned to output a structured latent space quickly and correctly. Figure 6 shows some sample reconstruction losses. Figure 7 shows the rapid decrease and stabilization of loss after only a dozen or so episodes of randomly explored games. This success can likely be attributed to the simple nature of the underlying state space, with relatively few states being possible compared to larger, more complicated games.

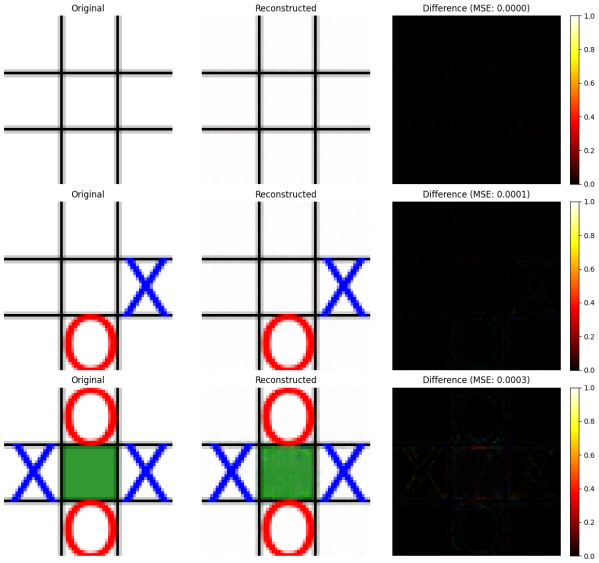

Fig. 6. Autoencoder reconstructions.

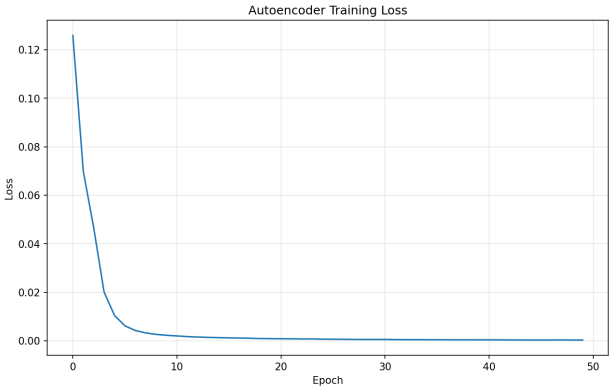

Fig. 7. Autoencoder losses.

2) Hybrid Policy: Likewise, the secondary policy was successful in training. While it needed far more episodes to stabilize, the secondary policy successfully learned to move the anomaly off the screen as quickly as possible so as to maximize future expected rewards from winning the resumed game of Tic-Tac-Toe. Given the small action space and simple model, it was expected to see the secondary policy perform well. Figure 8 shows the training rewards collected by the agent, where some failures to move the anomaly (0 reward) are expected due to the exploration term in training and the possibility of the anomaly never appearing.

In validation, the hybrid policy was found to be just as successful as the nominal model at winning Tic-Tac-Oh-No. Since the only means of performing worse are to fail to encode information in the latent space or fail to move the anomaly off the screen, this validates the

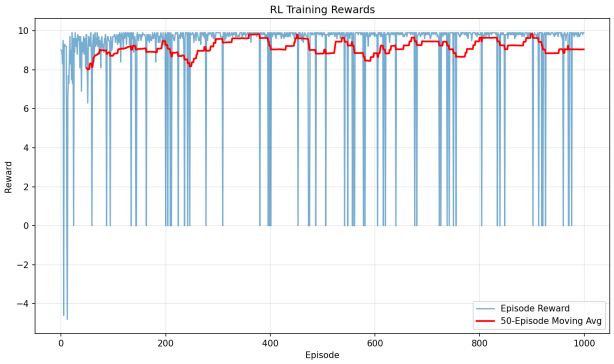

Fig. 8. Secondary policy training rewards.

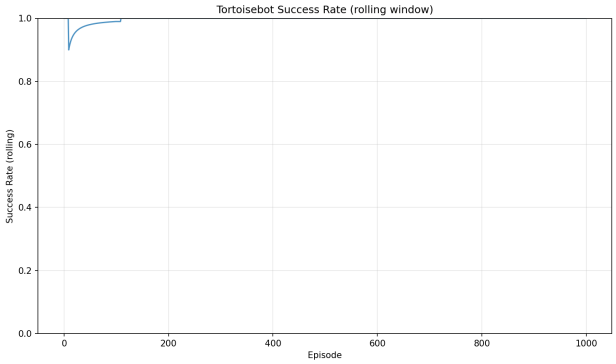

Fig. 10. Augmented policy success rate.

success of the encoder and the secondary policy.

B. Tortoisebot

In Figure 9 and 10, we can see the augmented policy learned to adjust the nominal controller's actions to better match the next state expected by the simulated environment. While it quickly learned enough to always reach the goal state, defined as distance within some threshold, the loss curve did not settle for around 800 episodes, far more than could be considered "quick" training in a real world system.

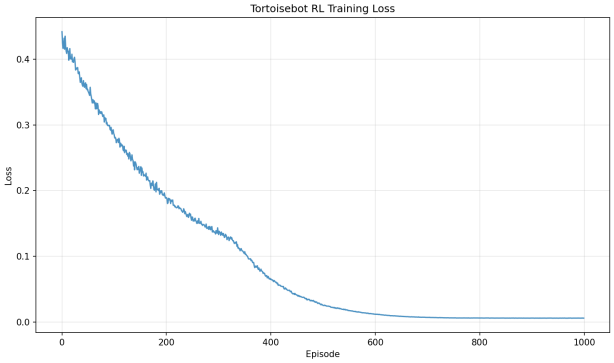

Fig. 9. Augmented policy loss.

In Figure 11, we compare what an unaugmented trajectory looks like compared to an augmented one at the same point in time. Note that the grey trajectory in each plot represents the idealized trajectory had the robot been acting in the simulated model up until that point. We find that the while augmented controller does indeed result in a different trajectory to the goal state, the middle of the trajectory is not noticeably different in its relative deviation from the simulated model.

Where we see the best performing results are in the overall success of the episodes as seen in validation.

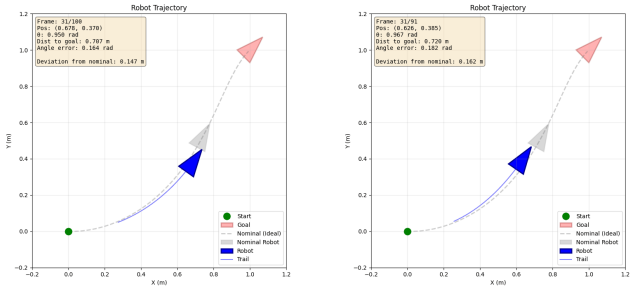

Fig. 11. An unaugmented trajectory (left) compared to an augmented trajectory (right)

Table I shows that the augmented controller was always successful in bringing the robot to the desired goal, doing so in significantly fewer steps with a smaller final error. Along each trajectory, the deviation from s'_{nom}, taken as the simple norm between the reached next state and the expected next state, is slightly reduced by the augmented controller with less variance. We interpret this to mean that the controller was most successful in helping realize states with smaller velocities closer to the start and goal states, thus reaching the goal more often and with better accuracy. It was marginally successful in reducing the deviation from the nominal model's expected next states along the way, but not enough to make a significant difference.

	Nominal Controller	Augmented Controller
Success Rate	0.32	1.0
Steps to completion	335 $\pm$ 115	149 $\pm$ 53
Final Pos Error	0.050 $\pm$ 0.379	0.013 $\pm$ 0.007
Final Theta Error	0.990 $\pm$ 0.164	0.080 $\pm$ 0.054
s'_{nom} Deviation	0.063 $\pm$ 0.115	0.054 $\pm$ 0.039

TABLE I
TOTAL REWARDS OF EACH POLICY

VI. Conclusion

In the course of two experiments, we investigate several ideas for adapting policies derived from simplified models into more complicated environments. The solutions in Tic-Tac-Oh-No demonstrate how a mismatch between the state and action space can be accounted for with a structured latent space, adapting the problem into one familiar to the nominal policy and factoring out unfamiliar states to a separate network, leading to successful play of the expanded game. The tortoisebot environment demonstrated augmenting a controller designed for overly simplified dynamics compared to the environment it was deployed it. The augmentations successfully allowed the controller to reach goals states it was unable to without assistance.

The structured latent space design leaves several areas for future investigation. While this work depended on a supervised learning method to structure the features of the latent space, the promising work in disentangling latent spaces gives opportunity for the encoder to be trained in tandem with the fixed nominal policy and to-be-trained secondary policy, perhaps discarding the decoder as done in [8]. In a similar manner of investigation, this work could lead to learning how to factorize complicated environments into simple, tractable parts. This leads to similar avenues explored by hierarchical networks or "skill-based" systems, but perhaps with an unsupervised approach this hierarchy could develop naturally and without explicit structure.

Mimicking the expected realizations of a simulated environment could lead to future investigations of techniques to bridge the sim-to-real gap. Augmenting a policy to conform to its expectations of the world approaches the sim-to-real gap in an almost backwards way; that is, making the "real" look like the sim from the perspective of the unaugmented policy. Looking at how close this "augmented environment" is to the original simulated model could be an interesting direction of this work, leading to bisimulation literature. A real-world hardware deployment of these ideas could also prove out their practicality in achieving that bridge.

VII. Contributions & Release

Ryan worked on the project alone barring the assistance disclosed in the acknowledgements. The off-the-shelf library Pytorch was used to create and train the networks.

The author grants permission for this report to be posted publicly.

Acknowledgment

AI was used to assist with writing Pytorch and plotting code. As an exercise in constant time hyperparameter tuning, AI also omnisciently provided hyperparameter values for training the networks. It did fine but a human was still needed to tune a couple of the networks, resulting in an exponential time complexity in tuning w.r.t. the human's perception of the passage of time (or so it felt).

References

[1] Y. Jun, J. Park, K. Choo, T. E. Choi, and S. J. Hwang, "Disentangling disentangled representations: Towards improved latent units via diffusion models," in *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2025, pp. 3559–3569.

[2] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

[3] X. Chen, Y. Duan, R. Houthooft, J. Schulman, I. Sutskever, and P. Abbeel, "Infogan: Interpretable representation learning by information maximizing generative adversarial nets," *Advances in neural information processing systems*, vol. 29, 2016.

[4] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.

[5] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[6] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, "beta-vae: Learning basic visual concepts with a constrained variational framework," in *International conference on learning representations*, 2017.

[7] L. Le, A. Patterson, and M. White, "Supervised autoencoders: Improving generalization performance with unsupervised regularizers," *Advances in neural information processing systems*, vol. 31, 2018.

[8] A. Zhang, R. McAllister, R. Calandra, Y. Gal, and S. Levine, "Learning invariant representations for reinforcement learning without reconstruction," *arXiv preprint arXiv:2006.10742*, 2020.

[9] S. Lange and M. Riedmiller, "Deep auto-encoder neural networks in reinforcement learning," in *The*

2010 International Joint Conference on Neural Networks (IJCNN), 2010, pp. 1–8.

[10] M. Watter, J. Springenberg, J. Boedecker, and M. Riedmiller, "Embed to control: A locally linear latent dynamics model for control from raw images," *Advances in neural information processing systems*, vol. 28, 2015.

[11] C. Gelada, S. Kumar, J. Buckman, O. Nachum, and M. G. Bellemare, "Deepmdp: Learning continuous latent space models for representation learning," in *International conference on machine learning*. PMLR, 2019, pp. 2170–2179.

[12] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *International conference on machine learning*. PMLR, 2019, pp. 2555–2565.

[13] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 3803–3810.

[14] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual policy learning," *arXiv preprint arXiv:1812.06298*, 2018.

[15] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *2019 international conference on robotics and automation (ICRA)*. IEEE, 2019, pp. 6023–6029.